COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/23/2026	**DATE SUBMITTED:**	10/23/2026
NAME:	Stephen Kurt Pellazar D Jay Ines	**INSTRUCTOR:**	Engr. JimLord M. Quejado

ACTIVITY NO. 14.1 **[Hands-on Activity 14.1 Algorithm Complexity]**

1. PROCEDURE OUTPUT

What procedures did you perform?

Procedure

Tabulated below, are a number of functions against n (from 1 to 10)

```
A = (log2 n) {log to base 2 of n}
B = n {linear in n}
C = (* n (log2 n)) {n log n}
D = (* n n) {quadratic in n}
E = (* n n n) {cubic in n}
F = (expt 2 n) {exponential in n}
G = (expt 3 n) {exponential in n}
H = (fact n) {factorial in n}
```

n	A	B	C	D	E	F	G	H
1	0.0	1	0.0	1	1	2	3	1
2	1.0	2	2.0	4	8	4	9	2
3	1.0	3	4.0	9	27	8	27	6
4	2.0	4	8.0	16	64	16	81	24
5	2.0	5	11.0	25	125	32	243	120
6	2.0	6	15.0	36	216	64	729	720
7	2.0	7	19.0	49	343	128	2187	5040
8	3.0	8	24.0	64	512	256	6561	40320
9	3.0	9	28.0	81	729	512	19683	362880
10	3.0	10	33.0	100	1000	1024	59049	3628800

Think of this as algorithms A through H with complexities as defined above, showing growth rate versus input size n. Tabulated below are functions F, G and H from above (ie 2 to power n, 3 to power n, and n factorial). Problem size n varies from 10 to 100 in steps of 10. I have assumed that we have a machine that can perform "the principle activity of the algorithm" in a micro second (ie. if we are considering a graph colouring algorithm, it can compare the colour of two nodes in a millionth of a second). The columns give the number of years this machine would take to execute those algorithms on problems of size n (note: YEARS). This is expressed as 10^x, "10 raised to the power x".

n	F	G	H
10	10^-10	10^-8	10^-6
20	10^-7	10^-3	10^4
30	10^-4	10^0	10^18
40	10^-1	10^5	10^34
50	10^1	10^10	10^50
60	10^4	10^15	10^68
70	10^7	10^19	10^86
80	10^10	10^24	10^105
90	10^13	10^29	10^124
100	10^16	10^34	10^144

Therefore, if we have a problem of size (lets say 40) and the machine specified above, if the best algorithm is O(2**n) it will take 1 year, if the best algorithm is O(3**n) it will take 100,000 years, and if the best algorithm is O(n!) it will take

10,000,000,000,000,000,000,000,000,000,000,000 years

approximately. Out of interest, the age of the universe is estimated to be between 15 and 20 billion years old, ie 20,000,000,000 years. That is, even at modest values of n we are presented with problems that will never be solved. It is almost tempting to say, that from a computational perspective, the universe is a small thing.

Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

Output: Ines and Pellazar

Bubble sort: algo 1
Merge sort: algo 2

	Best Case	**Worst Case**	**Analysis**
Algo 1	O(n), exits early, no swaps, one pass	O(n^2)	Comparison n = 100 -> 4950, n = 100 -> 49,995,00. Grows 100x for every 10x in n.
Algo 2	O(n log n)	O(n log n)	Space O(n) for the temp array. Roughly n log n, here is the comparison: ~664, ~9,966, ~132,877. Cost grows a little over 10x per 10x in n.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1	Algorithm 2
Best Case	n=100: 0.52 µs; n=1,000: 5.00 µs; n=10,000: 49.41 µs	n=100: 9.54 µs; n=1,000: 137.58 µs; n=10,000: 1.73 ms
Worst Case	n=100: 75.78 µs; n=1,000: 8.17 ms; n=10,000: 753.81 ms	n=100: 9.17 µs; n=1,000: 158.97 µs; n=10,000: 1.67 ms

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Supplementary Pellazar

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.
For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis

```
Algorithm LinearSearch(arr, target):
    Input: An array 'arr' of size N, target value 'target'
    Output: Index of 'target' if found, else -1

    For i from 0 to length(arr) - 1 do:
        If arr[i] == target then:
            Return i
    Return -1
```

- Time and Space Complexities

Metric	Linear Search	Binary Search
Best-Case Time Complexity	O(1)(Target is at the first position)	O(1)(Target is at the exact middle)
Average-Case Time Complexity	O(N)	O(Log N)
Worst-Case Time Complexity	O(N)(Target is at the end or	O(Log N) (Target is at the

	absent)	boundaries or absent)
Space Complexity	O(1)(Iterative, auxiliary space)	O(1) (Iterative approach)
Data Requirement	Unsorted or Sorted arrays	Strictly **Sorted** arrays

For the empirical analysis, the deliverable must be the filled-out table and your analysis

Input Size (N)	**Elapsed Time for Binary Search (Worst-Case)**	**Elapsed Time for Linear Search (Worst-Case)**
1,000	~0.001 ms	~0.05 ms
10,000	~0.002 ms	~0.50 ms
100,000	~0.004 ms	~5.20 ms

8. Conclusion

Provide the following:

Summary of lessons learned

- Linear search checks items one by one, which makes it slow for big datasets. Binary search is much faster because it keeps cutting the search area in half. However, linear search works on any list, while binary search only works if the list is already sorted.

Analysis of the procedure

- To test the worst-case time, we had to search for an item that was not in the list so the computer had to do the maximum amount of work. We also needed precise timers because binary search finishes so quickly that regular clocks cannot easily measure it.

Analysis of the supplementary activity

- Testing these code runtimes shows how theory works in real life. As the input size grew from 1,000 to 100,000, linear search became noticeably slower, while binary search stayed fast and barely took any extra time.

Supplementary Ines

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.
For the theoretical analysis, the deliverable must be:

Pseudocode Analysis:
BinarySearch(A, n, target)
lo = 0, hi = n-1

```
while lo <= hi
mid = (lo + hi) / 2
if A[mid] == target: return mid
else if A[mid] < target: lo = mid + 1
else: hi = mid - 1
return -1
```

Time and Space Complexities

	BEST	WORST	SPACE
LINEAR	O(1)	O(n)	O(1)
BINARY	O(1)	O(log n)	O(1)

For the empirical analysis, the deliverable must be the filled-out table and your analysis

Input Size (N)	**Elapsed Time for Binary Search (Worst-Case)**	**Elapsed Time for Linear Search (Worst-Case)**
1,000	0.068us	3.64us
10,000	0.072us	31.17us
100,000	0.100us	327.85us

Provide your analysis

- Linear search time grew about 10x each time N grew 10x. That is the O(n) behavior
- Binary Search time barely moved (0.068 -> 0.100 µs) even though N grew 100×. Its comparisons only went from 9 to 16, which matches $\log_2 N$.
- At N = 100,000, Binary Search was about 3,300× faster than Linear Search, and the gap keeps widening as N grows

8. Conclusion

Provide the following:

Summary of lessons learned

- In this activity I learned that the complexity of an algorithm measures how its time and space needs grow as the input size n grows. I learned that the dominant term and the input size matter most, so coefficients and hardware only change the units. I also learned the difference between best, worst, and average case, and that problems like $O(2^n)$ and O(n!) become intractable even for small n, while polynomial algorithms stay practical.

Analysis of the procedure

- I compared Bubble Sort and Merge Sort at n = 100, 1,000, and 10,000. Theoretically, Bubble Sort is O(n) in the best case and $O(n^2)$ in the worst case with O(1) space, while Merge Sort is O(n log n) in both cases with O(n) space. The experiments matched this. Bubble Sort on sorted data was the fastest result (49.41 µs at n = 10,000), but on reverse-sorted data it took 753.81 ms, about 450 times slower

than Merge Sort's 1.67 ms. Merge Sort's best and worst times were almost the same, which shows it behaves predictably. The comparison counts also matched the formulas, such as 49,995,000 for Bubble Sort's worst case at n = 10,000.

Analysis of the supplementary activity

- I compared Linear Search (O(n)) and Binary Search (O(log n)), both with O(1) space. Linear Search time grew about 10× for every 10× increase in N (3.64 µs to 327.85 µs), while Binary Search stayed almost flat (0.068 µs to 0.100 µs) because it only needed 9 to 16 comparisons. At N = 100,000, Binary Search was about 3,300 times faster. The tradeoff is that Binary Search requires sorted data, so sorting first only pays off if many searches will be done.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CHATGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar corrections and ideas**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to correct grammars and used like a tutor bot.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.